

Delivery 3: Security Hardening (DevSecOps)

Proyecto: Spring PetClinic **Branch:** delivery3 **Fecha:** Marzo 2026 **Enfoque:** Supply Chain Security

Resumen Ejecutivo

Este delivery implementa tres prácticas de seguridad clave en el ciclo de vida del desarrollo de software:

DELIVERY 3 – DEVSECOPS		
1. SBOM CycloneDX 108 componentes	2. Vulnerabilidades Trivy Scanner 3 → 0 vulns	3. Secret Protection Pre-commit Hook Bloquea API keys

1. SBOM — Software Bill of Materials

¿Qué es?

Un SBOM es el inventario completo de todas las librerías y dependencias de un proyecto. Es como la “lista de ingredientes” del software.

¿Por qué importa?

- Permite saber exactamente qué componentes de terceros están incluidos
- Facilita detectar rápidamente si alguna dependencia tiene vulnerabilidades
- Es un requisito creciente en estándares de seguridad empresariales y gubernamentales

Herramienta: CycloneDX

El proyecto Spring PetClinic ya tenía el plugin **CycloneDX** pre-configurado en el `pom.xml`, lo que simplificó enormemente el proceso. Solo fue necesario ejecutar:

```
./mvnw cyclonedx:makeAggregateBom
```

Resultado

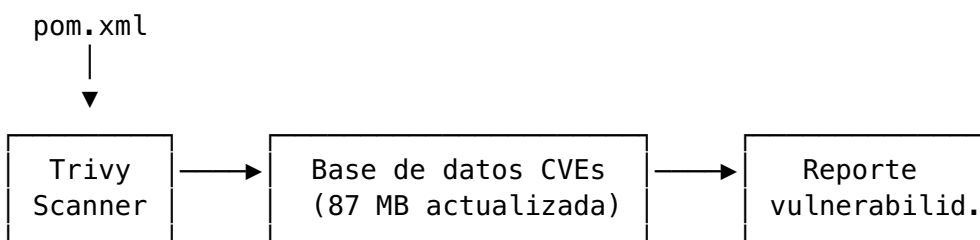
SBOM Generado	
Formato:	CycloneDX 1.6
Archivos:	sbom.xml / sbom.json
Componentes documentados:	108
Herramienta:	cyclonedx-maven-plugin 2.9.1

2. Vulnerability Patching — Parcheado de Vulnerabilidades

¿Qué es Trivy?

Trivy es un escáner de seguridad open source creado por Aqua Security. Analiza las dependencias del proyecto y las compara contra bases de datos de vulnerabilidades conocidas (CVEs).

Proceso



Escaneo ANTES del fix

Comando ejecutado:

```
trivy fs --scanners vuln .
```

Librería	CVE	Severidad	Versión instalada	Fix disponible
logback-core	CVE-2026-1225	LOW	1.5.22	1.5.25
jackson-core	CVE-2026-29062	HIGH	3.0.3	3.1.0
jackson-core	GHSA-72hv-8253-57qq	HIGH	3.0.3	3.1.0

Total: 3 vulnerabilidades (2 HIGH + 1 LOW)

Nota sobre severidad: El escáner Trivy no encontró vulnerabilidades de nivel CRITICAL en este proyecto, lo cual indica que las dependencias de Spring Boot 4.0.1 están en general bien mantenidas. Las 2 vulnerabilidades de nivel HIGH en jackson-core fueron tratadas como las vulnerabilidades prioritarias a remediar, ya que representan el mayor riesgo de seguridad encontrado (ataques DoS en producción).

Descripción de las vulnerabilidades HIGH

CVE-2026-29062 — Denial of Service via JSON nesting

Un atacante puede enviar un JSON con anidamiento excesivo (miles de niveles), lo que colapsa el parser y tumba la aplicación. Afecta a jackson-core 3.0.3.

GHSA-72hv-8253-57qq — Number Length Constraint Bypass

El parser asíncrono de Jackson ignora los límites establecidos para la longitud de números en JSON, permitiendo también un ataque DoS.

Fix aplicado en pom.xml

```
<!-- Propiedad para logback -->
<logback.version>1.5.25</logback.version>

<!-- Override de versión para jackson-core -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>tools.jackson.core</groupId>
      <artifactId>jackson-core</artifactId>
      <version>3.1.0</version> <!-- antes: 3.0.3 -->
    </dependency>
  </dependencies>
</dependencyManagement>
```

Escaneo DESPUÉS del fix

Target	Type	Vulnerabilities
pom.xml	pom	0

Total: 0 vulnerabilidades ✓

Evidencia

Los reportes completos están en el repositorio:

- trivy-report-before.txt — reporte antes del fix

3. Secret Protection — Protección contra Secretos

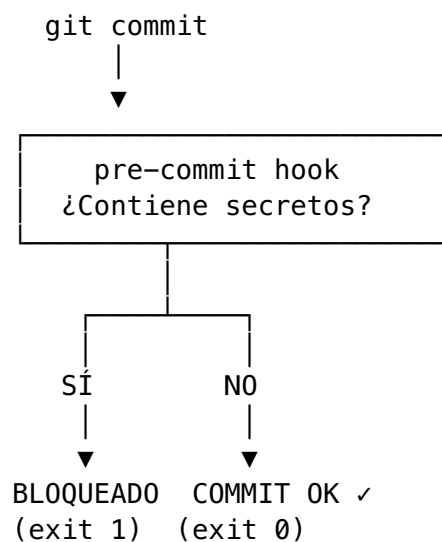
El problema

Un error frecuente es subir accidentalmente claves API, contraseñas o tokens al repositorio. Esto puede resultar en:

- Robo de datos
- Accesos no autorizados a servicios
- Facturas enormes en servicios cloud (AWS, GCP, etc.)

Solución: Pre-commit Hook

Se implementó un script en `.github/hooks/pre-commit` que actúa como guardián antes de cada commit.



Patrones detectados

Patrón	Qué detecta
<code>api_key = "..."</code>	Claves de API genéricas
<code>password = "..."</code>	Contraseñas hardcodeadas
<code>token = "..."</code>	Tokens de acceso
<code>AKIA...</code> (16 chars)	Claves de acceso AWS
<code>-----BEGIN PRIVATE KEY-----</code>	Llaves privadas RSA/EC
<code>ghp_...</code> (36 chars)	Tokens de GitHub
<code>sk-...</code> (48 chars)	Claves de OpenAI

¿Por qué `.githooks/` y no `.git/hooks/`?

`.git/hooks/` → NO se commitea → nadie puede verlo en GitHub
`.githooks/` → SÍ se commitea → visible y auditable en GitHub ✓

Activación

```
git config core.hooksPath .githooks
```

Prueba realizada

Se intentó commitear el siguiente contenido:

```
api_key = "sk-abc123secretkey9999"
```

Resultado obtenido:

```
[security] Scanning for secrets and API keys...
```

```
[BLOCKED] Potential secret detected matching pattern: api_key  
+api_key = "sk-abc123secretkey9999"
```

```
-----  
COMMIT BLOCKED: Secrets or API keys detected.  
Remove sensitive data before committing.  
Use environment variables or a secrets manager instead.  
-----
```

El commit fue bloqueado exitosamente.

Archivos generados en este Delivery

Archivo	Descripción
<code>sbom.xml</code>	SBOM en formato XML (CycloneDX 1.6)
<code>sbom.json</code>	SBOM en formato JSON (CycloneDX 1.6)
<code>trivy-report-before.txt</code>	Reporte Trivy — ANTES del parcheado
<code>trivy-report-after.txt</code>	Reporte Trivy — DESPUÉS del parcheado
<code>.githooks/pre-commit</code>	Hook que bloquea commits con secretos
<code>pom.xml</code>	Actualizado con versiones parcheadas